

parent: SPEC-001

Data Model

Overview

The system has 5 core data structures stored as JSON files in the project directory. The key design principle is that **the edit document is a thin reference layer** — it contains only source IDs and range references. All timing, text, and metadata are resolved at runtime by joining the edit document against the transcripts or scene index.

Two data layers, independently populated

Every source can have **both** layers — they are not mutually exclusive:

Layer	Atom	Produced by	Provides
Transcript	Word	ar-edit transcribe (whisper.cpp)	What is being <i>said</i> — word text, timing, confidence
Scene Index	Scene	ar-edit index (ffmpeg scene detect + agent descriptions)	What is being <i>shown</i> — setting, subjects, action

An interview video has transcript AND visual context. A drone shot has scenes AND maybe ambient audio worth transcribing. Search spans both layers.

A shot in the edit document specifies its range using **one of** these:

- 1. **Word range** — from_word / to_word (requires transcript)
- 2. **Scene range** — from_scene / to_scene (requires scene index)
- 3. **Time range** — from_ms / to_ms (direct, always works)

This means an agent can find a segment by searching the transcript ("when Alice mentions climate policy"), the scene descriptions ("exterior shot of the coastline"), or both ("the part where Alice is standing outside discussing the reef").

```
project/
├─ manifest.json           # Project manifest
├─ sources/               # Symlinks or copies of source videos
│   ├── src-001.mp4
│   └── src-002.mp4
├─ transcripts/
│   ├── src-001.transcript.json
│   └── src-002.transcript.json
├─ index/
│   ├── src-001.index.json
│   └── src-002.index.json
├─ thumbnails/
│   ├── src-001_00m00s.jpg
│   ├── src-001_00m10s.jpg
│   └── ...
└─ edits/
```

```
|— rough-cut.edit.json
|— final.edit.json
```

1. Project Manifest

The root of the project. Created by `ar-edit init`.

```
{
  "version": "1.0.0",
  "name": "my-project",
  "created": "2026-02-19T12:00:00Z",
  "sources": [
    {
      "id": "src-001",
      "path": "sources/src-001.mp4",
      "original_filename": "interview-alice.mp4",
      "duration_ms": 124500,
      "video_codec": "h264",
      "audio_codec": "aac",
      "resolution": [1920, 1080],
      "frame_rate": 29.97,
      "audio_channels": 2,
      "audio_sample_rate": 48000,
      "added": "2026-02-19T12:01:00Z",
      "transcribed": true,
      "indexed": true
    }
  ],
  "next_source_id": 2,
  "defaults": {
    "whisper_model": "base",
    "thumbnail_interval_sec": 10,
    "render_codec": "h264",
    "render_container": "mp4"
  }
}
```

Key points:

- `next_source_id` ensures IDs are never reused
- `path` is always relative to project root
- Source metadata is extracted from ffprobe on `ar-edit add`

2. Transcript

Produced by `ar-edit transcribe`. This is the central data structure — everything else references into it via word indices.

```
{
  "source_id": "src-001",
  "model": "base",
  "language": "en",
  "duration_ms": 124500,
  "segments": [
    {
      "index": 0,
      "start_ms": 0,
```

```

    "end_ms": 5230,
    "text": "Welcome to the interview today we're going to talk about",
    "words": [
      {
        "index": 0,
        "text": "Welcome",
        "start_ms": 0,
        "end_ms": 420,
        "confidence": 0.95
      },
      {
        "index": 1,
        "text": "to",
        "start_ms": 420,
        "end_ms": 540,
        "confidence": 0.97
      },
      {
        "index": 2,
        "text": "the",
        "start_ms": 540,
        "end_ms": 650,
        "confidence": 0.98
      }
    ]
  },
  {
    "index": 1,
    "start_ms": 5230,
    "end_ms": 11800,
    "text": "the impact of climate policy on regional communities",
    "words": [
      {
        "index": 9,
        "text": "the",
        "start_ms": 5230,
        "end_ms": 5400,
        "confidence": 0.96
      }
    ]
  }
],
"word_count": 487
}

```

Key points:

- **Word indices are global and sequential across the entire transcript** (not per-segment). Word 0 is the first word of the video. Word 486 is the last.
- `start_ms` / `end_ms` give millisecond-precision boundaries for each word
- `segments` are whisper.cpp's natural utterance groupings — useful for display but the edit system operates on **word indices**
- `confidence` is whisper.cpp's per-token probability — useful for the agent to judge transcript reliability

Mapping: whisper.cpp output → internal format

whisper.cpp's `--output-json` produces tokens with timestamps. The ingestion step:

1. Filters to word tokens only (strips special tokens like `[BLANK]` , `[SOT]` , etc.)
2. Assigns sequential global word indices
3. Converts timestamp strings ("00:00:05,230") to integer milliseconds
4. Preserves segment boundaries

3. Edit Document

The edit document is intentionally minimal. It is an **ordered list of shots**, where each shot is a reference into a transcript.

```
{
  "name": "rough-cut",
  "version": 1,
  "created": "2026-02-19T13:00:00Z",
  "modified": "2026-02-19T13:45:00Z",
  "next_shot_id": 6,
  "shots": [
    {
      "id": "shot-001",
      "source": "src-001",
      "range": { "words": { "from": 0, "to": 52 } }
    },
    {
      "id": "shot-002",
      "source": "src-003",
      "range": { "words": { "from": 200, "to": 280 } }
    },
    {
      "id": "shot-003",
      "source": "src-002",
      "range": { "scenes": { "from": 0, "to": 2 } }
    },
    {
      "id": "shot-004",
      "source": "src-001",
      "range": { "words": { "from": 100, "to": 142 } }
    },
    {
      "id": "shot-005",
      "source": "src-002",
      "range": { "time": { "from_ms": 15000, "to_ms": 22000 } }
    }
  ]
}
```

Key points:

- **range** is a tagged union — exactly one of `words`, `scenes`, or `time`
 - `words` — indices into the source's transcript (requires transcript)
 - `scenes` — indices into the source's scene index (requires index with descriptions)
 - `time` — direct millisecond timestamps (always works, no prerequisites)
- Shot IDs are stable — removing shot-002 does NOT renumber shot-003
- `next_shot_id` ensures IDs are never reused
- The same source can appear in multiple shots with different range types
- You can mix speech and B-roll freely in the same edit
- **No resolved data stored** — timing/text is always resolved at runtime

Resolved View (computed, not stored)

When you run `ar-edit edit show`, the system joins the edit document against transcripts and/or scene indices to produce:

```
{
  "name": "rough-cut",
  "total_duration_ms": 52200,
  "shots": [
    {
      "id": "shot-001",
```

```

    "source": "src-001",
    "range_type": "words",
    "start_ms": 0,
    "end_ms": 12400,
    "duration_ms": 12400,
    "text_preview": "Welcome to the interview today we're going to talk about the impact of...",
    "scene_preview": null
  },
  {
    "id": "shot-002",
    "source": "src-003",
    "range_type": "words",
    "start_ms": 67300,
    "end_ms": 82100,
    "duration_ms": 14800,
    "text_preview": "What we found in the regional assessment was that communities...",
    "scene_preview": null
  },
  {
    "id": "shot-003",
    "source": "src-002",
    "range_type": "scenes",
    "start_ms": 0,
    "end_ms": 18000,
    "duration_ms": 18000,
    "text_preview": null,
    "scene_preview": "Aerial drone shot of coastline at sunset, waves breaking on reef"
  },
  {
    "id": "shot-005",
    "source": "src-002",
    "range_type": "time",
    "start_ms": 15000,
    "end_ms": 22000,
    "duration_ms": 7000,
    "text_preview": null,
    "scene_preview": "Close-up of coral formation underwater"
  }
]
}

```

The critical insight: the edit document is just pointers. Transcripts and scene indices are the source of truth for timing and content. The resolved view merges both layers for display.

4. Source Index

Produced by `ar-edit index`. Combines metadata, transcript summary, and thumbnail references into a searchable structure.

```

{
  "source_id": "src-001",
  "indexed_at": "2026-02-19T12:05:00Z",
  "metadata": {
    "duration_ms": 124500,
    "resolution": [1920, 1080],
    "codec": "h264",
    "file_size_bytes": 52428800
  },
  "thumbnails": [
    {
      "path": "thumbnails/src-001_00m00s.jpg",
      "timestamp_ms": 0,
      "description": null
    }
  ],
}

```

```

{
  "path": "thumbnails/src-001_00m10s.jpg",
  "timestamp_ms": 10000,
  "description": null
},
{
  "path": "thumbnails/src-001_00m18s.jpg",
  "timestamp_ms": 18000,
  "description": "Scene change detected"
}
],
"scene_count": 4,
"scenes": [
  {
    "index": 0,
    "start_ms": 0,
    "end_ms": 18000,
    "thumbnail": "thumbnails/src-001_00m00s.jpg",
    "description": "Interior office, wide shot – two people seated at desk, window with city skyline"
  },
  {
    "index": 1,
    "start_ms": 18000,
    "end_ms": 45000,
    "thumbnail": "thumbnails/src-001_00m18s.jpg",
    "description": "Close-up of speaker A (Alice), bookshelf background"
  },
  {
    "index": 2,
    "start_ms": 45000,
    "end_ms": 78000,
    "thumbnail": "thumbnails/src-001_00m45s.jpg",
    "description": "Close-up of speaker B (interviewer), nodding"
  },
  {
    "index": 3,
    "start_ms": 78000,
    "end_ms": 124500,
    "thumbnail": "thumbnails/src-001_01m18s.jpg",
    "description": null
  }
]
}

```

Key points:

- `description` fields are `null` by default — designed to be populated by an LLM agent via `ar-edit index update-description`
- `scenes` are detected via ffmpeg scene change filter (`select='gt(scene,0.3)'`)
- Thumbnails are extracted at both fixed intervals AND scene changes
- The index is a read-side structure — it never affects edits or renders

Agent-populated descriptions

An LLM agent would:

1. Read the index JSON
2. Read each thumbnail image
3. Write descriptions back: `ar-edit index set-description src-001 --timestamp 18000 --text "Interior office, two people at a desk, window with city view"`

5. Annotated Transcript (for text-based editing)

Exported by `ar-edit transcripts export --format editable`. This is a Markdown file with HTML comment annotations that survive text editing.

```
# Source: src-001 - interview-alice.mp4
```

```
<!-- ar-edit:src-001:w0 -->Welcome<!-- /ar-edit --> <!-- ar-edit:src-001:w1 -->to<!-- /ar-edit -->
<!-- ar-edit:src-001:w2 -->the<!-- /ar-edit --> <!-- ar-edit:src-001:w3 -->interview<!-- /ar-edit -->
<!-- ar-edit:src-001:w4 -->today<!-- /ar-edit --> <!-- ar-edit:src-001:w5 -->we're<!-- /ar-edit -->
...
---
```

```
# Source: src-002 - interview-bob.mp4
```

```
<!-- ar-edit:src-002:w0 -->So<!-- /ar-edit --> <!-- ar-edit:src-002:w1 -->the<!-- /ar-edit -->
<!-- ar-edit:src-002:w2 -->first<!-- /ar-edit --> <!-- ar-edit:src-002:w3 -->thing<!-- /ar-edit -->
...
```

When the user deletes text or reorders sections, the HTML comments travel with the words. `ar-edit edit from-transcript` parses the remaining annotations to reconstruct source/word-index pairs.

This format is verbose by design. An alternative compact format wraps at the segment level:

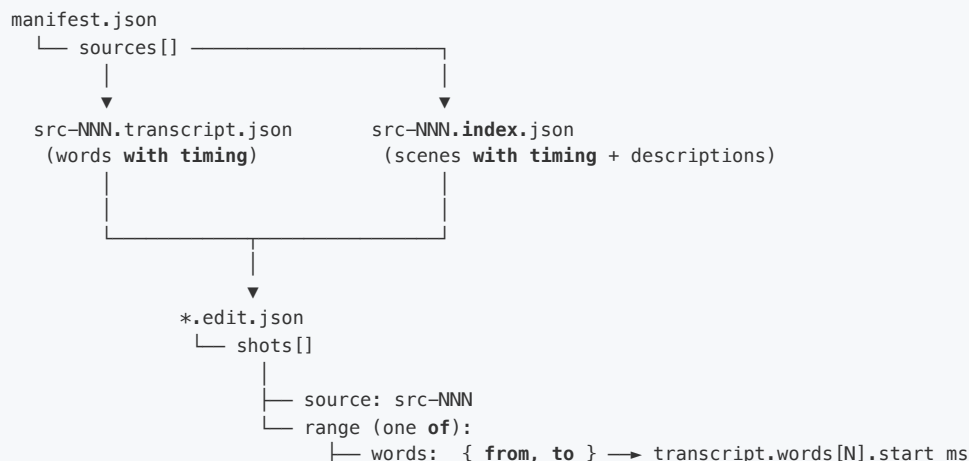
```
# Source: src-001 - interview-alice.mp4
```

```
<!-- ar-edit:src-001:w0-w8 -->
Welcome to the interview today we're going to talk about
<!-- /ar-edit:src-001 -->

<!-- ar-edit:src-001:w9-w18 -->
the impact of climate policy on regional communities
<!-- /ar-edit:src-001 -->
```

The segment-level format is the default. The user edits by deleting, reordering, or splitting these blocks. If they split a block mid-sentence, `ar-edit edit from-transcript` falls back to fuzzy text matching against the transcript to resolve word boundaries.

Relationships



```
└─ scenes: { from, to } → index.scenes[N].start_ms
└─ time:    { from_ms, to_ms } (direct)
```

The edit document is a **join table**. It has no data of its own — only references. This means:

- Transcripts can be regenerated (better model) without touching edits
- Scene descriptions can be updated without touching edits
- Edits are tiny (a 100-shot edit is ~2KB)
- Validation is a simple bounds check against the relevant layer
- The agent can construct edit documents by reasoning over either transcripts, scene descriptions, or both
- A single edit can mix speech and visual sources freely

Rust Type Summary

```
// manifest.json
struct Manifest {
    version: String,
    name: String,
    created: DateTime<Utc>,
    sources: Vec<Source>,
    next_source_id: u32,
    defaults: Defaults,
}

struct Source {
    id: String,           // "src-001"
    path: PathBuf,        // relative to project root
    original_filename: String,
    duration_ms: u64,
    video_codec: String,
    audio_codec: String,
    resolution: (u32, u32),
    frame_rate: f64,
    audio_channels: u8,
    audio_sample_rate: u32,
    added: DateTime<Utc>,
    transcribed: bool,
    indexed: bool,
}

// src-NNN.transcript.json
struct Transcript {
    source_id: String,
    model: String,
    language: String,
    duration_ms: u64,
    segments: Vec<TranscriptSegment>,
    word_count: u32,
}

struct TranscriptSegment {
    index: u32,
    start_ms: u64,
    end_ms: u64,
    text: String,
    words: Vec<Word>,
}

struct Word {
    index: u32,           // global sequential index
    text: String,
    start_ms: u64,
```

```

        end_ms: u64,
        confidence: f32,
    }

    // *.edit.json
    struct EditDocument {
        name: String,
        version: u32,
        created: DateTime<Utc>,
        modified: DateTime<Utc>,
        next_shot_id: u32,
        shots: Vec<Shot>,
    }

    struct Shot {
        id: String,           // "shot-001"
        source: String,       // "src-001"
        range: ShotRange,
    }

    /// Tagged union – exactly one variant per shot
    enum ShotRange {
        /// References word indices in transcript (inclusive)
        Words { from: u32, to: u32 },
        /// References scene indices in source index (inclusive)
        Scenes { from: u32, to: u32 },
        /// Direct millisecond timestamps (no prerequisite)
        Time { from_ms: u64, to_ms: u64 },
    }

    // src-NNN.index.json
    struct SourceIndex {
        source_id: String,
        indexed_at: DateTime<Utc>,
        metadata: SourceMetadata,
        thumbnails: Vec<Thumbnail>,
        scene_count: u32,
        scenes: Vec<Scene>,
    }

    struct Thumbnail {
        path: PathBuf,
        timestamp_ms: u64,
        description: Option<String>,
    }

    struct Scene {
        index: u32,           // sequential, like word indices
        start_ms: u64,
        end_ms: u64,
        thumbnail: PathBuf,
        description: Option<String>,
    }

```